

Web-приложение психологического аудита команд

0. Суть продукта в одном абзаце

Web-приложение — *корпоративный портал*, где руководитель видит результаты психологического аудита своей команды и команд ниже по иерархии. На каждую команду готовится 4 артефакта (тепловая карта, рейтинг, готовность команды, мотивационная карта). Консультант загружает готовые артефакты, руководитель их просматривает. Навигация рекурсивная: клик по подчинённому, у которого есть своя команда, открывает такой же портал уровнем ниже — на любую глубину. Персональные данные защищены шифрованием на уровне API.

1. Функционал (полное описание)

1.1 Роли

Роль	Видит	Может
Консультант / Администратор	Всех клиентов и всю иерархию	Заводить клиента, строить иерархию, загружать артефакты, выдавать доступы
Руководитель-заказчик	Свой узел иерархии и всё поддерево ниже	Просматривать артефакты, проваливаться вниз по поддереву
Сотрудник (фаза 2)	Только свои данные	— (вне MVP, заложено архитектурно)

Принцип доступа: запрещено всё, что не разрешено явно (deny-by-default). Минимальные привилегии. Раскрытие ФИО — отдельное право.

1.2 Артефакты (4 типа на каждую команду)

1. Тепловая карта — подчинённые в таблице по иерархии потенциала; ячейки кликабельны и ведут к провалу на уровень ниже.
2. Рейтинг — оценка компетенций, поведения, стилей; основные тезисы + дашборд + рейтинги по команде.
3. Готовность команды — потенциал сотрудников под ключевые задачи; тезисы + дашборд + управленческие задачи.

4. Мотивационная карта — мотиваторы и демотиваторы по каждому сотруднику + предлагаемые действия.

Каждый артефакт привязан к узлу иерархии (команде) и имеет версию (под будущую «динамику во времени»).

1.3 Главное меню (портал)

- Шапка: хлебные крошки*(навигационная цепочка вверх экрана) (Компания → Команда ГД → ... → текущая команда) и кнопка «вверх».
- Левая колонка: фото и ФИО (или код) текущего руководителя, его должность; ниже — список его команды. У сотрудника со своей командой — маркер-стрелка (клик = провал ниже).
- Правое пространство: 4 крупные панели перехода к артефактам.

1.4 Навигация (рекурсия)

- Портал команды — один переиспользуемый экран, работающий на любом уровне.
- Клик по подчинённому со своей командой → тот же портал, но для его команды (уровень ниже).
- Глубина не ограничена: уровней столько, сколько команд в иерархии клиента.
- Хлебные крошки и кнопка «вверх» дают возврат и не дают потеряться в глубине.

1.5 Функциональные требования (MVP)

- FR-1. Консультант создаёт клиента и строит дерево команд (глубина не ограничена; в MVP тестируем на 2–3 уровнях).
- FR-2. Консультант загружает/обновляет 4 артефакта на узел.
- FR-3. Консультант выдаёт руководителю доступ на узел (= корень его зоны видимости).
- FR-4. Руководитель видит и навигирует только своё поддерево.
- FR-5. Портал отображает профиль, команду и 4 панели; панели открывают артефакты.
- FR-6. По умолчанию — обезличенно (коды). ФИО — только при наличии права, по запросу.
- FR-7. Каждое раскрытие ФИО логируется (кто, какой код, когда, какой узел).
- FR-8. Повторная публикация артефакта создаёт новую версию.

1.6 Вне MVP (бэклог для фокус-группы)

Фильтры и поиск, сравнение команд, экспорт в PDF/презентацию, заметки руководителя, динамика во времени, опросы и автоскоринг внутри приложения, роль сотрудника.

2. Пользовательский путь (полное описание)

2.1 Консультант — публикация

1. Вход и аутентификация.
2. Выбор/создание клиента.
3. Построение дерева команд.
4. Выбор узла → загрузка 4 артефактов.
5. Выдача руководителю доступа на узел.
6. Готово: артефакты опубликованы, по умолчанию обезличены.

2.2 Руководитель — просмотр (рекурсивный)

1. Вход и аутентификация.
2. Система определяет зону видимости (корень = выданный узел).
3. Открывается портал команды: профиль слева, команда списком, 4 панели справа.
4. Руководитель открывает любую из 4 панелей и изучает артефакт.
5. Клик по подчинённому со своей командой → тот же портал уровнем ниже (рекурсия).
6. Возврат — через хлебные крошки или кнопку «вверх».
7. При наличии права на ФИО — запрос к API, проверка права и зоны, показ имён, запись в аудит. Иначе — показ по кодам.

3. Системная архитектура

3.1 Принцип

Своего сервера нет — приложение, база и API целиком в облаке вендора. Поскольку «географически» отделить ФИО некуда, переданные защищаются криптографически: чувствительные поля (ФИО и привязки) хранятся в зашифрованном виде, а ключ шифрования управляется отдельным сервисом и не лежит рядом с данными. API отдаёт ФИО расшифрованными только по авторизованному запросу; во всех прочих случаях наружу идут только коды.

3.2 Логические компоненты

- Фронтенд (портал) — UI, оперирует кодами; ФИО получает только при авторизованном запросе и не кэширует их.
- Бэкенд/API — проверка аутентификации, прав и зоны видимости на каждый запрос; обращение к хранилищу; аудит.
- Хранилище данных — иерархия, артефакты, коды; чувствительные поля зашифрованы (шифрование на уровне поля).

- Сервис ключей (KMS) — хранит и выдаёт ключи шифрования; отделён от хранилища данных.
- Журнал аудита — фиксирует доступы и раскрытия ФИО.

3.3 Как защищаются перданные

- Шифрование на покое: ФИО и привязки «код → человек» шифруются на уровне поля (а не только диск целиком).
- Ключи отдельно от данных: ключ в KMS облака; компрометация базы без ключа не раскрывает ФИО.
- Шифрование в канале: TLS между фронтом, API и хранилищем.
- Расшифровка по требованию: API расшифровывает ФИО только при наличии права и в пределах зоны видимости запрашивающего.
- Аудит: каждое раскрытие ФИО логируется.
- Минимизация: фронтенд не хранит и не кэширует ФИО; по умолчанию везде коды.

3.4 Угрозы и как закрываем (мини-threat-model)

- Компрометация хранилища → ФИО зашифрованы, ключ в KMS → имена не раскрываются.
- Обход доступа (увидеть чужое поддерево) → проверка зоны видимости на сервере, не на фронте.
- Утечка через кэш/экспорт → экспорта в MVP нет; ФИО не кэшируются.
- Инсайдер тянет лишние ФИО → отдельное право + полный аудит раскрытий.
- Перехват трафика → TLS на всех каналах.

4. Реализация: через что и на каком языке

Так как всё разворачивается в облаке вендора, есть два реалистичных класса решений. Оба укладываются в «развернуть полностью в облаке».

4.1 Вариант 1 — открытая low-code платформа в облаке (приоритетный)

Платформы: Appsmith, ToolJet, Budibase (open-source, self-hostable). Их разворачивают контейнером (Docker) в облаке вендора — то есть приложение и данные остаются под вашим контролем внутри арендованного облака, без передачи стороннему SaaS.

- Что кодить: визуальная сборка экранов + вставки на JavaScript для логики (условия раскрытия ФИО, обращения к API). Полноценного «писать весь фронт руками» не требуется.
- Язык: JavaScript для логики внутри платформы; SQL для запросов к хранилищу.
- Плюсы: данные не уходят в чужой SaaS, есть RBAC и аудит-логи из коробки, нет vendor lock-in (открытый код), шифрование полей реализуемо на уровне API/БД.
- Минусы: нужно корректно развернуть контейнер и настроить окружение.

4.2 Вариант 2 — проприетарная no-code платформа (быстрый прототип)

Платформа: Bubble и аналоги — приложение и данные на инфраструктуре вендора, хостинг/SSL/масштабирование «из коробки».

- Что кодить: визуально + workflow-логика; сложная авторизация и шифрование требуют плагинов или внешнего API.
- Плюсы: максимальная скорость до первого прототипа.
- Минусы: нет экспорта исходного кода (lock-in: «перестали платить — приложение офлайн»), переданные физически в SaaS вендора, тонкая настройка шифрования полей ограничена.

4.3 Вариант 3 — vibecode / классический стек

Фронтенд на React (или Vue) + бэкенд (Node.js/TypeScript или Python) + БД (PostgreSQL) + KMS облака. Разворачивается в облаке вендора как контейнеры.

- Язык: TypeScript/JavaScript (фронт и бэк) или Python (бэк); SQL.
- Плюсы: максимальная гибкость UI (портал «как у вас в голове»), полный контроль шифрования полей и ключей.
- Минусы: это уже разработка, не no-code; дольше и дороже первой версии.

4.5 Что обязательно учесть при реализации (чек-лист)

- Шифрование ФИО на уровне поля + ключи в KMS, отдельно от данных.
- API проверяет право и зону видимости на каждый запрос (серверная авторизация).
- Аудит каждого раскрытия ФИО.
- По умолчанию наружу только коды; ФИО — по требованию и не кэшируются.
- TLS на всех каналах; секреты/ключи — вне кода.
- Версионирование артефактов под будущую динамику.
- Рекурсивная навигация + хлебные крошки + ограничение зоны видимости.

Функции

1. Авторизация

Назначение:

Предназначена для авторизации пользователя в системе.

Сценарий:

1. Пользователь вводит логин / пароль;
2. Система сверяет данные пользователя;
3. Система производит авторизацию пользователя (либо отказывает в доступе).

Результат выполнения:

Пользователь авторизован в приложении с нужным набором прав.

2. Заполнение/редактирование OBS компании

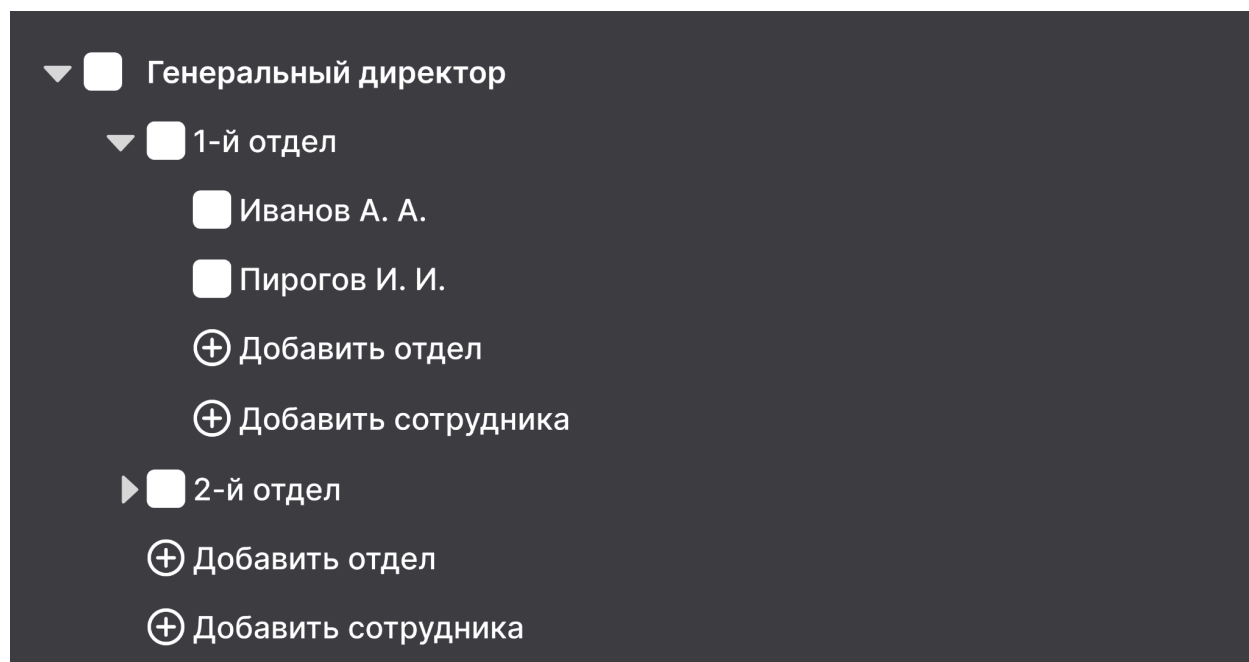
Назначение:

Предназначена для создания и редактирования структуры компании в веб приложении. Интерфейс должен выглядеть как конструктор создания квадратиков с должностями в них.

Возможное решение для режима создания - древовидная структура в формате (сотрудники в отделы заполняются отдельно):

- Отдел 1:
 - Отдел 2:
 - Отдел 21;
 - Отдел 22.
 - Отдел 3.
 - Отдел 4.

Образно говоря вот так (см. картинку):



Сценарий:

1. После авторизации в соответствующей вкладке пользователь должен нажать на “Создать OBS” или “Редактировать” (если уже есть).
2. Ввести данные организации (отделы + сотрудники в них).
3. Сохранить результат.
4. Система должна отобразить созданную / отредактированную структуру компании в формате дерева.

Результат выполнения:

Создана структура компании, отражена в соответствующей вкладке.

3. Загрузка артефакта

Назначение:

Предназначена для загрузки / ввода данных артефакта к команде. Есть два варианта реализации (будет обсуждаться):

1. (a) Простая - просто грузится pdf в соответствующее окно (плохо подходит для просмотра с телефона).
2. (b) Сложная - создаем уникальную форму ввода для каждого артефакта. После ввода трансформируем под нужную верстку - веб / мобайл.

Сценарий:

1. Пользователь переходит к OBS компании.

2. Выбирает нужную команду.
3. Загружает артефакт:
 - a. Выбирает нужный артефакт, далее грузит в систему pdf этого артефакта.
 - b. Выбирает нужный артефакт, Система отображает уникальную форму ввода. Пользователь заполняет форму и сохраняет.

Результат выполнения:

Артефакт приложен к соответствующей команде.

4. Просмотр артефактов команды

Примечание:

Текущему пользователю для выбора доступны только те команды, которые находятся в его подчинении или на уровне ниже,

Назначение:

Предназначена для просмотра артефактов команды, приложенных в систему ранее. Есть два варианта реализации (будет обсуждаться):

1. (a) Простая - просмотр ранее приложенного pdf.
2. (b) Сложная - просмотр адаптированной ранее введенной формы.

Сценарий:

1. Пользователь переходит в раздел OBS.
2. Выбирает интересующую команду.
3. В окне команды видит доступные артефакты (для выбора другой команды требуется вернуться на экран OBS).
4. Выбирает нужный артефакт:
 - a. Видит приложенный ранее pdf.
 - b. Видит в Системе форму просмотра артефакта, созданную при введении данных артефакта.

Результат выполнения:

Пользователь просмотрел артефакт команды.

5. Профиль пользователя

Назначение:

Предназначена для создания профиля, отображения в нем информации о компании, смене пароля (в случае особой авторизации).

Сценарий:

1. Пользователь переходит в раздел “Профиль”.
2. Видит инфу о компании, учетке.
3. Нажимает на “Редактировать” около поля пароля.
4. Вводит старый и новый пароли, нажимает на сохранить.

Результат выполнения:

Профиль изучен, пароль изменен.